

Building Generative AI-Powered Applications with Python

Maulana Hafidz Ismail

21 Oktober 2025

Contents

1	Image Captioning with Generative AI	1
1.1	Model-model generatif AI yang menjadi dasar dalam pengembangan aplikasi generatif AI	1
1.1.1	Variational Autoencoders (VAEs)	1
1.1.2	Generative Adversarial Networks (GANs)	1
1.1.3	Transformer-based Models	1
1.1.4	Diffusion Models	2
1.2	Definisi Foundation Model	2
1.3	Karakteristik Foundation Models	2
1.4	Kemampuan Foundation Models	2
1.5	Contoh Foundation Models	2
1.6	Adaptasi Foundation Models	3
1.7	Keterbatasan Foundation Models	3
1.8	Pengenalan Image Captioning AI	3
1.9	Hugging Face Platform	3
1.10	Hugging Face Transformers	4
1.11	Pengenalan Bootstrapping Language-Image Pretraining (BLIP)	4
1.12	Menggunakan BLIP pada Hugging Face	5
1.13	Pengenalan Gradio	6
1.13.1	Mulai Menggunakan Gradio	7
1.13.2	Klasifikasi Gambar dengan PyTorch	9
2	Create Your Own ChatGPT-Like Website	12
2.1	Pengantar Chatbot	12
2.2	Komponen Utama Chatbot	12
2.3	Pemilihan LLM	12
2.4	Pengembangan Chatbot	12
2.5	Tokenizer	12
3	Create a Voice Assistant	13
3.1	Komponen Utama Project	13
3.2	Definisi dan Konsep Dasar Docker	13
3.3	Proses dan Teknologi Dasar Docker	13
3.4	Manfaat Docker	13
3.5	Tantangan Docker	13
3.6	IBM Watson speech-to-text (STT)	14
3.7	IBM Watson text-to-speech (TTS)	14
3.8	Mengintegrasikan STT dan TTS Watson dalam Proyek Kecerdasan Buatan	15
3.9	Masa depan teknologi yang didukung suara	15

4	Generative AI-Powered Meeting Assistant	17
4.1	Teknologi yang Digunakan	17
4.2	IBM Watsonx.ai	17
4.3	Produk dalam IBM Watsonx	17
4.4	Alat dan Fitur dalam watsonx.ai	17
4.5	Pengenalan ke Meta Llama 2	18
4.5.1	Fitur Kunci dari Meta Llama 2	18
4.5.2	Limitasi Meta Llama 2	19
4.6	Aplikasi dari Meta Llama 2	19
4.7	Wawasan Teknis terkait Meta Llama 2	20
4.7.1	Contoh 1: Membuat teks berdasarkan prompt	20
4.7.2	Contoh 2: Ringkasan data	20
4.8	Pengenalan ke OpenAI Whisper	21
4.8.1	Fitur Kunci dari Whisper	21
4.9	Menyiapkan Whisper	21
4.9.1	Contoh: Transkripsi Dasar dengan Whisper	22
4.9.2	Use Case Nyata	22
4.9.3	Fitur Tambahan	23
4.10	Mengintegrasikan Whisper ke dalam Aplikasi	23
5	Summarize Your Private Data with Generative AI and RAG	24
5.1	Pengenalan LLMs dan Generative AI	24
5.2	Proyek Chatbot	24
5.3	Teknologi yang Digunakan	24
5.4	Tujuan dan Manfaat LangChain	24
5.5	Cara Kerja LangChain	25
5.6	Aplikasi Praktis LangChain	25
5.7	Pengenalan Retrieval-Augmented Generation (RAG)	25
5.8	Tantangan LLM	25
5.9	Solusi RAG	25
5.10	Manfaat RAG	26
5.11	Manfaat menggunakan Llama 2 pribadi dengan RAG	26
5.11.1	Alasan menggunakan Llama 2 pribadi untuk RAG	26
5.12	LangChain untuk mengimplementasikan RAG	26
5.12.1	Proses implementasi RAG dengan Llama 2	27
6	Babel Fish (Universal Language Translator) with LLM and SST TTS	28
7	Build and AI Career Coach	29

1 Image Captioning with Generative AI

1.1 Model-model generatif AI yang menjadi dasar dalam pengembangan aplikasi generatif AI

1.1.1 Variational Autoencoders (VAEs)

VAEs adalah model generatif yang digunakan untuk mengurangi dimensi data dan menghasilkan versi baru dari data tersebut. Dan fiturnya sebagai berikut:

- Bekerja dengan berbagai jenis data seperti gambar, teks, dan audio.
- Menggunakan encoder untuk mempelajari distribusi probabilitas dari data input dan menyimpannya dalam latent space, yang merupakan representasi terkompresi dari data.

1.1.2 Generative Adversarial Networks (GANs)

GANs adalah model yang terdiri dari dua jaringan saraf konvolusional (CNN) yang bersaing satu sama lain untuk menghasilkan data yang realistis. Fiturnya sebagai berikut:

- Satu CNN berfungsi sebagai generator yang menghasilkan sampel data, sementara yang lain berfungsi sebagai discriminator yang membedakan antara sampel nyata dan palsu.
- Dapat digunakan untuk menghasilkan gambar baru, melakukan transfer gaya, dan bahkan membuat deep fakes.

1.1.3 Transformer-based Models

Model berbasis transformer adalah arsitektur yang dirancang untuk mengatasi masalah vanishing gradients pada RNN, dengan menggunakan mekanisme perhatian untuk memfokuskan pada bagian penting dari teks. Berikut fiturnya:

- Mampu memodelkan ketergantungan jangka panjang dalam teks.
- Digunakan dalam pemrosesan bahasa alami dan dapat menghasilkan teks yang koheren dan relevan secara kontekstual.

Recurrent Neural Network (RNN) adalah jenis jaringan saraf yang dirancang untuk memproses data sekuensial dengan cara yang mempertahankan informasi dari langkah sebelumnya. Ini berarti RNN dapat "mengingat" informasi dari input sebelumnya dan menggunakannya untuk mempengaruhi output saat ini. Informasi lain terkait RNN:

- Memori Jangka Pendek: RNN memiliki kemampuan untuk menyimpan informasi dari langkah sebelumnya, yang membuatnya cocok untuk tugas-tugas seperti analisis teks, pengenalan suara, dan prediksi urutan.

- Vanishing Gradient Problem: Namun, RNN sering mengalami masalah yang disebut vanishing gradients, di mana informasi dari langkah-langkah yang lebih awal dalam urutan menjadi semakin sulit untuk dipertahankan saat jaringan dilatih. Ini adalah salah satu alasan mengapa model berbasis transformer dikembangkan.

1.1.4 Diffusion Models

Model difusi adalah model generatif yang mengatasi penurunan informasi akibat noise dalam latent space dengan menerapkan prinsip difusi. Berikut fiturnya:

- Melibatkan dua langkah: forward diffusion (menambahkan noise) dan reverse diffusion (menghilangkan noise untuk memulihkan data).
- Contoh model ini termasuk OpenAI's Dall-E2 dan Stability AI's Stable Diffusion XL, yang menghasilkan konten grafis berkualitas tinggi.

1.2 Definisi Foundation Model

Foundation Model adalah model yang dilatih pada sejumlah besar data dan dapat disesuaikan untuk berbagai aplikasi. Ini adalah model umum yang memiliki kemampuan multimodal atau dapat menerima banyak bentuk input seperti gambar, teks, suara, dll.

1.3 Karakteristik Foundation Models

- Pre-training: Proses di mana model dilatih menggunakan algoritma tanpa pengawasan pada data yang tidak diberi label, memungkinkan model untuk menghubungkan informasi yang beragam secara mandiri.
- Multimodal: Kemampuan untuk menerima input dalam berbagai format seperti teks, gambar, audio, atau video.

1.4 Kemampuan Foundation Models

- Generative AI: Semua foundation models memiliki kemampuan generatif, tetapi tidak semua model generatif adalah foundation models.
- Large Language Models (LLMs): Foundation models yang dilatih pada basis data pemrosesan bahasa alami yang besar, seperti GPT-3 dan GPT-4.

1.5 Contoh Foundation Models

- OpenAI's Dall-E: Model yang dapat melakukan berbagai tugas terkait gambar.
- Google's Bert: Model yang dilatih pada parameter besar untuk pemrosesan bahasa.

1.6 Adaptasi Foundation Models

Foundation models dapat disesuaikan untuk banyak aplikasi, memungkinkan bisnis kecil untuk membuat model AI generatif yang lebih efisien dan terjangkau.

1.7 Keterbatasan Foundation Models

- **Bias:** Output yang dihasilkan dapat terpengaruh oleh bias dalam data pelatihan.
- **Hallucination:** LLMs dapat menghasilkan informasi yang salah karena salah menginterpretasikan konteks data.

1.8 Pengenalan Image Captioning AI

- Image Captioning AI: Teknologi yang mengubah informasi visual dari gambar menjadi bahasa yang dapat dibaca mesin. Ini membantu dalam meningkatkan aksesibilitas, hasil pencarian, dan keamanan.
- Manfaat:
 - Memungkinkan penemuan konten yang lebih dalam, meningkatkan kehadiran di media sosial, dan manajemen data yang efisien.
 - Bertujuan untuk membangun alat AI yang tidak hanya melihat gambar, tetapi juga memahaminya dan membuat file teks yang memberikan deskripsi bermakna tentang isi gambar.
 - Dengan memberikan deskripsi yang tepat, pencarian gambar menjadi lebih mudah dan mengurangi beban kerja.
- BLIP (Bootstrapping Language-Image Pre-training): Model yang dapat melakukan berbagai tugas multi-modal, termasuk pengambilan gambar-teks dan captioning gambar.

1.9 Hugging Face Platform

Hugging Face adalah platform open source untuk kecerdasan buatan (AI) yang memungkinkan kolaborasi antara ilmuwan, pengembang, dan bisnis untuk membangun alat machine learning yang dipersonalisasi.

Platform ini bertujuan untuk menciptakan pusat bagi komunitas AI open source untuk berbagi model, dataset, dan aplikasi, sehingga AI dapat diakses oleh semua jenis pengguna.

Hugging Face memiliki lebih dari 250,000 open models, 50,000 datasets, dan satu juta open demos yang terus berkembang.

Platform ini menyediakan library transformer open source dengan lebih dari 25,000 pretrained models untuk PyTorch, TensorFlow, dan Google Jax.

- PyTorch: Library deep learning yang digunakan untuk membangun model AI.

- TensorFlow: Platform machine learning yang digunakan untuk membangun dan melatih model AI.
- Google Jax: Framework machine learning yang memungkinkan komputasi yang lebih efisien.

1.10 Hugging Face Transformers

Hugging Face Transformers adalah perpustakaan sumber terbuka yang populer yang menyediakan model dan alat pemrosesan bahasa alami (NLP) terkini. Perpustakaan ini menawarkan berbagai model yang telah dilatih sebelumnya untuk berbagai tugas NLP, termasuk klasifikasi teks, jawaban pertanyaan, dan terjemahan bahasa.

Salah satu fitur utama Hugging Face Transformers adalah dukungannya untuk pembelajaran multimodal, yang menggabungkan data teks dan gambar untuk tugas-tugas seperti penandaan gambar dan jawaban pertanyaan visual. Kemampuan ini sangat relevan dalam pembahasan Bootstrapping Language-Image Pretraining (BLIP), karena memanfaatkan data teks dan gambar untuk meningkatkan pemahaman dan kemampuan generasi deskripsi gambar oleh model AI.

Dalam pembahasan ini, kita akan menjelajahi cara menggunakan Hugging Face Transformers, khususnya model BLIP, untuk penulisan keterangan gambar dalam Python. Kita akan menunjukkan cara memuat model yang telah dilatih sebelumnya, memproses gambar, dan menghasilkan keterangan, sekaligus memperlihatkan kemampuan perpustakaan ini dalam menjembatani kesenjangan antara bahasa alami dan konten visual.

1.11 Pengenalan Bootstrapping Language-Image Pretraining (BLIP)

BLIP mewakili kemajuan signifikan di persimpangan antara pemrosesan bahasa alami (NLP) dan penglihatan komputer. BLIP, yang dirancang untuk meningkatkan model kecerdasan buatan (AI), memperkuat kemampuannya dalam memahami dan menghasilkan deskripsi gambar. Sistem ini belajar mengasosiasikan gambar dengan teks yang relevan, sehingga dapat menghasilkan keterangan gambar, menjawab pertanyaan terkait gambar, dan mendukung pencarian berbasis gambar.

BLIP penting untuk beberapa alasan:

- Pemahaman yang lebih mendalam: Ini memberikan pemahaman yang lebih rinci tentang konten dalam gambar, melampaui pengenalan objek untuk memahami adegan, tindakan, dan interaksi.
- Pembelajaran multimodal: Dengan mengintegrasikan data teks dan gambar, BLIP memfasilitasi pembelajaran multimodal, yang lebih mirip dengan cara manusia memandang dunia.

- Aksesibilitas: Membuat deskripsi gambar yang akurat dapat membuat konten lebih mudah diakses oleh orang dengan gangguan penglihatan.
- Pembuatan konten: Ini mendukung upaya kreatif dan pemasaran dengan menghasilkan teks deskriptif untuk konten visual, menghemat waktu, dan meningkatkan kreativitas.

Aplikasi praktis BLIP adalah dalam pengembangan sistem penandaan foto otomatis. Sistem semacam ini dapat digunakan di berbagai bidang. Sistem ini meningkatkan platform media sosial dengan menyarankan keterangan foto secara otomatis untuk foto yang diunggah. Sistem ini juga membantu sistem manajemen aset digital dengan menyediakan deskripsi yang dapat dicari untuk gambar yang disimpan.

1.12 Menggunakan BLIP pada Hugging Face

Hugging Face menyediakan platform untuk bereksperimen dengan BLIP dan model AI lainnya. Berikut adalah contoh cara menggunakan BLIP untuk penandaan gambar (image captioning) dalam Python.

Pastikan Anda telah menginstal Python dan perpustakaan Transformers. Jika belum, Anda dapat menginstal perpustakaan Transformers menggunakan pip. Lihat kode berikut.

```
# Install the transformers library
!pip install transformers Pillow torch torchvision
torchaudio

from transformers import BlipProcessor,
    BlipForConditionalGeneration
from PIL import Image

# Initialize the processor and model from Hugging Face
processor = BlipProcessor.from_pretrained("Salesforce/blip-
    image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("
    Salesforce/blip-image-captioning-base")

# Load an image
image = Image.open("path_to_your_image.jpg")

# Prepare the image
inputs = processor(image, return_tensors="pt")

# Generate captions
outputs = model.generate(**inputs)
caption = processor.decode(outputs[0], skip_special_tokens=
    True)

print("Generated Caption:", caption)
```

BLIP juga dapat menjawab pertanyaan tentang isi sebuah gambar. Lihat kode berikut.

```
import requests
from PIL import Image
from transformers import BlipProcessor,
    BlipForConditionalGeneration

# Load BLIP processor and model
processor = BlipProcessor.from_pretrained("Salesforce/blip-
    image-captioning-large")
model = BlipForConditionalGeneration.from_pretrained("
    Salesforce/blip-image-captioning-large")

# Image URL
img_url = 'https://storage.googleapis.com/sfr-vision-
    language-research/BLIP/demo.jpg'
raw_image = Image.open(requests.get(img_url, stream=True).
    raw).convert('RGB')

# Specify the question you want to ask about the image
question = "What is in the image?"

# Use the processor to prepare inputs for VQA (image +
    question)
inputs = processor(raw_image, question, return_tensors="pt")

# Generate the answer from the model
out = model.generate(**inputs)

# Decode and print the answer to the question
answer = processor.decode(out[0], skip_special_tokens=True)
print(f"Answer: {answer}")
```

BLIP dari Hugging Face Transformers membuka peluang baru untuk aplikasi kecerdasan buatan (AI) dengan memungkinkan pemahaman yang lebih mendalam terhadap konten visual dan deskripsi teks. Dengan menggunakan BLIP, pengembang dan peneliti dapat menciptakan aplikasi yang lebih intuitif, mudah diakses, dan menarik yang menjembatani kesenjangan antara dunia visual dan bahasa alami.

1.13 Pengenalan Gradio

Gradio adalah cara untuk mendemonstrasikan Machine Learning Model Anda dengan antarmuka web yang ramah pengguna sehingga siapa pun dapat menggunakannya di mana saja. Gradio adalah open source Python package yang memungkinkan Anda dengan cepat membangun demo atau aplikasi web untuk model pembelajaran mesin, API, atau fungsi Python apa pun. Anda kemudian dapat membagikan tautan ke demo atau aplikasi web Anda menggunakan fitur

berbagi bawaan Gradio. Tidak diperlukan pengalaman dalam JavaScript, CSS, atau hosting web.

Alasan menggunakan Gradio:

- Kemudahan penggunaan: Gradio memungkinkan pembuatan antarmuka untuk model dengan hanya beberapa baris kode.
- Kekuatan adaptasi: Gradio mendukung berbagai jenis input dan output, seperti teks, gambar, file, dan lainnya.
- Berbagi dan kolaborasi: Antarmuka dapat dibagikan dengan orang lain melalui URL unik, memudahkan kolaborasi dan pengumpulan umpan balik.

1.13.1 Mulai Menggunakan Gradio

Untuk mulai menggunakan Gradio, Anda perlu menginstal perpustakaan terlebih dahulu. Anda dapat menginstal Gradio menggunakan pip:

```
pip install gradio
```

Kemudian inisialisasi Gradio:

```
import gradio as gr

def greet(name, intensity):
    return "Hello, " + name + "!" * int(intensity)

demo = gr.Interface(
    fn=greet,
    inputs=["text", "slider"],
    outputs=["text"],
)

demo.launch(server_name="127.0.0.1", server_port= 7860)
```

Perhatikan bahwa untuk membuat demo pertama Anda, Anda telah membuat instance dari kelas `gr.Interface`. Kelas Interface dirancang untuk membuat demo untuk model pembelajaran mesin yang menerima satu atau lebih input dan mengembalikan satu atau lebih output.

Kelas Interface memiliki tiga argumen inti:

- `fn`: Fungsi untuk membungkus antarmuka pengguna (UI) di sekitar
- `inputs`: Komponen Gradio yang akan digunakan untuk input. Jumlah komponen harus sesuai dengan jumlah argumen dalam fungsi Anda.
- `outputs`: Komponen Gradio yang akan digunakan untuk output. Jumlah komponen harus sesuai dengan jumlah nilai kembalian dari fungsi Anda.

Argumen `fn` bersifat fleksibel — Anda dapat melewatkan fungsi Python apa pun yang ingin Anda bungkus dengan UI. Dalam contoh di atas, Anda melihat fungsi yang relatif sederhana, tetapi fungsi tersebut dapat berupa apa saja, mulai dari generator musik hingga kalkulator pajak hingga fungsi prediksi dari model pembelajaran mesin yang telah dilatih sebelumnya.

Argumen input dan output menerima satu atau lebih komponen Gradio. Seperti yang akan kita lihat, Gradio menyertakan lebih dari 30 komponen bawaan (seperti komponen `gr.Textbox()`, `gr.Image()`, dan `gr.HTML()`) yang dirancang untuk aplikasi pembelajaran mesin.

Jika fungsi Anda menerima lebih dari satu argumen, seperti pada contoh di atas, berikan list komponen ke `inputs`, dengan setiap komponen input sesuai dengan salah satu argumen fungsi secara berurutan. Hal yang sama berlaku jika fungsi Anda mengembalikan lebih dari satu nilai: cukup berikan daftar komponen ke `outputs`. Fleksibilitas ini membuat kelas `Interface` menjadi cara yang sangat powerful untuk membuat demo. Mari kita buat antarmuka sederhana untuk model penandaan gambar. Model BLIP (Bootstrapped Language Image Pretraining) dapat menghasilkan teks keterangan untuk gambar. Berikut cara membuat antarmuka Gradio untuk model BLIP.

```
pip install transformers
pip install torch
```

```
import gradio as gr
from transformers import BlipProcessor,
    BlipForConditionalGeneration
from PIL import Image

processor = BlipProcessor.from_pretrained("Salesforce/blip-
    image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("
    Salesforce/blip-image-captioning-base")

def generate_caption(image):
    # Now directly using the PIL Image object
    inputs = processor(images=image, return_tensors="pt")
    outputs = model.generate(**inputs)
    caption = processor.decode(outputs[0],
        skip_special_tokens=True)
    return caption

def caption_image(image):
    """
    Takes a PIL Image input and returns a caption.
    """
    try:
        caption = generate_caption(image)
        return caption
    except Exception as e:
```

```

        return f"An error occurred: {str(e)}"

iface = gr.Interface(
    fn=caption_image,
    inputs=gr.Image(type="pil"),
    outputs="text",
    title="Image Captioning with BLIP",
    description="Upload an image to generate a caption."
)

iface.launch(server_name="127.0.0.1", server_port= 7860)

```

Di sini, kita menggunakan `BlipProcessor` dan `BlipForConditionalGeneration` dari pustaka `transformers` untuk membuat model picture captioning. Contoh ini menunjukkan cara membuat antarmuka web menggunakan Gradio, di mana parameter input menentukan gambar dan outputnya adalah teks caption yang dihasilkan. Parameter judul dan deskripsi memperkaya antarmuka dengan memberikan konteks dan petunjuk bagi pengguna.

Model penandaan gambar seperti BLIP merupakan alat yang sangat powerful di berbagai bidang, mulai dari membantu individu dengan gangguan penglihatan memahami konten gambar hingga mengorganisir dan mencari melalui perpustakaan foto besar dengan efisien. Membuat antarmuka Gradio untuk model semacam ini membuatnya dapat diakses oleh pengguna non-teknis untuk berinteraksi dan memanfaatkan teknologi ini. Misalnya, fotografer atau manajer aset digital dapat menggunakan aplikasi Anda untuk secara otomatis menghasilkan nama deskriptif untuk gambar mereka, meningkatkan kegunaan dan kemudahan pencarian dalam perpustakaan digital mereka.

1.13.2 Klasifikasi Gambar dengan PyTorch

Klasifikasi gambar merupakan tugas utama dalam pengolahan citra komputer. Pengembangan klasifikasi yang lebih baik untuk mengidentifikasi objek yang terdapat dalam sebuah gambar merupakan bidang penelitian yang aktif, karena memiliki aplikasi yang luas, mulai dari kendaraan otonom hingga pencitraan medis.

Model-model semacam ini sangat cocok digunakan bersama komponen input gambar Gradio. Dalam tutorial ini, kita akan membangun demo web untuk mengklasifikasikan gambar menggunakan Gradio. Seluruh aplikasi web dapat dibangun menggunakan Python.

Pertama, kita memerlukan model klasifikasi gambar. Untuk tutorial ini, kita akan menggunakan model Resnet-18 yang telah dilatih sebelumnya, karena model ini dapat diunduh dengan mudah dari PyTorch Hub. Anda dapat menggunakan model yang telah dilatih sebelumnya yang berbeda atau melatih model Anda sendiri.

```
import torch
```

```
model = torch.hub.load('pytorch/vision:v0.6.0', 'resnet18',
                        pretrained=True).eval()
```

Selanjutnya, kita perlu mendefinisikan fungsi yang menerima masukan pengguna, dalam hal ini berupa gambar, dan mengembalikan prediksi. Prediksi tersebut harus dikembalikan dalam bentuk dictionary, di mana key-nya adalah nama class dan value-nya adalah confidence probabilities. Nama-nama kelas akan dimuat dari berkas teks ini.

Dalam kasus model yang telah dilatih sebelumnya, bentuknya akan seperti ini:

```
import requests
from PIL import Image
from torchvision import transforms

# Download human-readable labels for ImageNet.
response = requests.get("https://git.io/JJkYN")
labels = response.text.split("\n")

def predict(inp):
    inp = transforms.ToTensor()(inp).unsqueeze(0)
    with torch.no_grad():
        prediction = torch.nn.functional.softmax(model(inp)[0],
                                                  dim=0)
        confidences = {labels[i]: float(prediction[i]) for i in
                       range(1000)}
    return confidences
```

Mari ulas, fungsinya mengambil satu parameter:

`inp`: the input image as a PIL image

Fungsi ini mengubah gambar input menjadi PIL Image dan kemudian menjadi tensor PyTorch. Setelah memproses tensor melalui model, fungsi ini mengembalikan prediksi dalam bentuk dictionary bernama `confidences`. Key-nya adalah label kelas, dan value-nya adalah confidence probabilities yang sesuai.

Pada bagian ini, kita mendefinisikan fungsi `predict` yang memproses gambar input untuk mengembalikan probabilitas prediksi. Fungsi ini pertama-tama mengubah gambar menjadi tensor PyTorch dan kemudian meneruskannya melalui model yang telah dilatih sebelumnya. Kami menggunakan fungsi `softmax` pada langkah akhir untuk menghitung probabilitas setiap kelas. Fungsi `softmax` sangat penting karena mengubah keluaran logit mentah dari model, yang dapat berupa angka real apa pun, menjadi probabilitas yang jumlahnya sama dengan 1. Hal ini memudahkan untuk menafsirkan keluaran model sebagai tingkat kepercayaan untuk setiap kelas.

Sekarang setelah fungsi prediktif kita sudah disiapkan, kita dapat membuat antarmuka Gradio untuknya.

Dalam hal ini, komponen input adalah komponen gambar seret dan lepas. Untuk membuat komponen ini, kita menggunakan `Image(type="pil")` yang membuat komponen dan menangani prapemrosesan untuk mengubahnya menjadi gambar PIL.

Komponen output akan berupa Label, yang menampilkan label teratas dalam bentuk yang rapi. Karena kita tidak ingin menampilkan semua 1.000 label kelas, kita akan menyesuaikannya untuk menampilkan hanya 3 kelas teratas dengan mengonfigurasinya sebagai `Label(num_top_classes=3)`.

Terakhir, kita akan menambahkan satu parameter lagi, parameter `examples`, yang memungkinkan kita untuk mengisi antarmuka dengan beberapa contoh yang telah ditentukan sebelumnya. Kode untuk Gradio terlihat seperti ini:

```
import gradio as gr
gr.Interface(fn=predict,
            inputs=gr.Image(type="pil"),
            outputs=gr.Label(num_top_classes=3),
            examples=["/content/lion.jpg", "/content/cheetah.jpg"]
            ).launch()
```

Contoh path sudah disediakan, `/content/lion.jpg` dan `/content/cheetah.jpg`, hanyalah tempat penampung. Anda harus mengganti paths ini dengan path aktual ke gambar-gambar di sistem atau server Anda tempat Anda menyimpan gambar-gambar yang ingin Anda gunakan untuk pengujian. Hal ini memastikan bahwa saat Anda atau orang lain menggunakan antarmuka Gradio, contoh-contoh tersebut dimuat dengan benar dan dapat digunakan untuk mendemonstrasikan fungsi klasifikasi gambar Anda.

Itu semua kode yang Anda butuhkan untuk membuat demo web untuk klasifikasi gambar. Jika Anda ingin berbagi dengan orang lain, coba atur `share=True` saat Anda menjalankan Interface.

Gradio memudahkan proses pembuatan demo web interaktif untuk model pembelajaran mesin. Dengan mengintegrasikan Gradio dengan model seperti BLIP untuk penandaan gambar, Anda dapat membuat aplikasi praktis dan ramah pengguna yang memanfaatkan kekuatan AI untuk menyelesaikan masalah dunia nyata. Alat ini tidak hanya membantu dalam menunjukkan kemampuan model Anda tetapi juga dalam mengumpulkan umpan balik berharga untuk perbaikan lebih lanjut.

2 Create Your Own ChatGPT-Like Website

2.1 Pengantar Chatbot

Chatbot adalah program komputer yang mensimulasikan percakapan manusia, memungkinkan pengguna untuk mendapatkan informasi dengan cepat.

Dengan teknologi AI, chatbot dapat memahami pertanyaan dan memberikan jawaban berdasarkan data yang telah dikumpulkan.

2.2 Komponen Utama Chatbot

- Transformer: Program yang berfungsi sebagai otak chatbot, mengelola pemrosesan data input dan output.
- LLM (Large Language Model): Model yang membantu chatbot memahami pertanyaan dan menghasilkan respons yang mirip dengan manusia.

2.3 Pemilihan LLM

Penting untuk memilih LLM yang sesuai dengan tujuan chatbot, seperti:

- GPT-2/GPT-3: Untuk generasi teks umum.
- BERT: Untuk analisis sentimen.
- T-5: Untuk terjemahan bahasa.

Pertimbangan lain dalam memilih LLM meliputi lisensi, ukuran model, data pelatihan, serta kinerja dan akurasi.

2.4 Pengembangan Chatbot

Anda akan menggunakan model BlenderBot dari Facebook, yang tersedia dengan lisensi sumber terbuka melalui Hugging Face.

Hugging Face's Python library transformers memudahkan interaksi dengan LLM dan mengoptimalkan input ke dalam format yang dapat dipahami oleh LLM, yaitu token.

2.5 Tokenizer

Tokenizer adalah komponen dalam Natural Language Processing (NLP) yang berfungsi untuk memecah teks menjadi unit-unit kecil yang disebut token. Token ini bisa berupa kata, potongan kata, atau karakter, tergantung jenis tokennya.

Token ini kemudian diubah menjadi representasi numerik agar model bisa memahaminya dan melakukan tugas seperti menjawab pertanyaan atau membuat teks.

3 Create a Voice Assistant

3.1 Komponen Utama Project

- GPT-3: Model yang memungkinkan asisten untuk memahami dan merepons input pengguna.
- IBM Watson Speech To Text (STT): Memungkinkan asisten untuk mendengar dan memahami respons pengguna.
- IBM Watson Text To Speech (TTS): Memungkinkan asisten untuk membacakan jawaban kembali kepada pengguna.

3.2 Definisi dan Konsep Dasar Docker

- Docker: Sebuah platform open-source untuk mengembangkan, mengirim, dan menjalankan aplikasi sebagai container. Container adalah unit standar perangkat lunak yang mengemas kode dan semua dependensinya.
- Container: Lingkungan terisolasi yang memungkinkan aplikasi berjalan secara konsisten di berbagai platform dan lingkungan.

3.3 Proses dan Teknologi Dasar Docker

- Namespaces: Fitur dari Linux kernel yang digunakan Docker untuk memberikan ruang kerja terisolasi yang disebut container. Setiap container memiliki set namespace sendiri, yang membatasi akses ke sumber daya.
- Go Programming Language: Bahasa pemrograman yang digunakan untuk menulis Docker.

3.4 Manfaat Docker

- Stabilitas: Lingkungan yang konsisten dan terisolasi menghasilkan penyebaran aplikasi yang stabil.
- Kecepatan: Penyebaran aplikasi dapat dilakukan dalam hitungan detik karena ukuran image Docker yang kecil dan dapat digunakan kembali.
- Automasi: Kemampuan automasi Docker membantu mengurangi kesalahan dan menyederhanakan siklus pemeliharaan.

3.5 Tantangan Docker

Docker tidak cocok untuk aplikasi yang memerlukan:

- High Performance: Kinerja tinggi yang mungkin tidak dapat dicapai dengan container.
- Security: Aplikasi yang memerlukan tingkat keamanan yang sangat tinggi.

- Monolithic Architecture: Aplikasi yang dibangun dengan arsitektur monolitik mungkin tidak optimal dalam penggunaan container.

3.6 IBM Watson speech-to-text (STT)

IBM Watson STT adalah layanan kecerdasan buatan (AI) yang mengubah bahasa lisan menjadi teks tertulis menggunakan teknik pembelajaran mesin canggih untuk mengembangkan aplikasi yang memerlukan masukan suara, seperti asisten suara yang dikendalikan suara, transkripsi rapat, atau meningkatkan dukungan pelanggan dengan perintah suara. Dalam STT, jaringan saraf dalam memproses sinyal audio untuk mentranskripsikan kata-kata yang diucapkan menjadi teks. Model-model ini dilatih menggunakan dataset yang beragam, termasuk berbagai bahasa, aksen, dan ucapan dalam lingkungan yang berbeda, untuk meningkatkan akurasi pengenalan dan fleksibilitas. STT menyediakan berbagai layanan untuk meningkatkan aplikasi kecerdasan buatan (AI):

- Pengenalan suara real-time: Watson STT dapat mentranskrip audio langsung saat sedang diucapkan, yang sangat penting untuk aplikasi interaktif.
- Dukungan bahasa dan dialek: STT mendukung berbagai bahasa dan dialek, menjadikannya fleksibel untuk aplikasi global.
- Penyesuaian: Pengguna dapat melatih layanan ini dengan terminologi dan pola ucapan khusus bidang, meningkatkan akurasi untuk aplikasi niche.

3.7 IBM Watson text-to-speech (TTS)

IBM Watson Text-to-Speech (TTS) melengkapi layanan STT dengan mengubah teks tertulis menjadi audio ucapan yang terdengar alami. Layanan TTS Watson termasuk di antara layanan terkemuka yang menghasilkan output suara yang realistis dan ekspresif. Teknologi TTS telah berkembang dari output suara yang sederhana dan terdengar robotik menjadi kemampuan menghasilkan ucapan yang mirip dengan nada dan intonasi manusia. Hal ini dicapai melalui model deep learning canggih yang memahami konteks teks, petunjuk emosional, dan nuansa linguistik. TTS menyediakan berbagai layanan untuk meningkatkan aplikasi kecerdasan buatan (AI):

- Suara yang ekspresif dan alami: TTS menyediakan berbagai pilihan suara dan bahasa yang membantu menghasilkan output sesuai dengan preferensi pengguna.
- Emosi dan ekspresivitas: TTS memungkinkan pengguna untuk mengontrol nada, emosi, dan ekspresivitas suara output agar sesuai dengan konteks percakapan.
- Penyesuaian: Seperti STT, Watson TTS memungkinkan penyesuaian suara dan dapat dilatih untuk menyertakan istilah teknis atau pelafalan khusus yang unik bagi suatu bisnis atau industri.

3.8 Mengintegrasikan STT dan TTS Watson dalam Proyek Kecerdasan Buatan

Mengintegrasikan layanan STT (Speech-to-Text) dan TTS (Text-to-Speech) ke dalam proyek AI dapat secara signifikan meningkatkan pengalaman pengguna dengan memfasilitasi interaksi yang alami dan intuitif.

Berikut adalah beberapa langkah dan pertimbangan untuk mengintegrasikan layanan suara IBM Watson ke dalam aplikasi:

- Kunci API dan pengaturan IBM Cloud: Layanan STT dan TTS dapat diakses melalui platform IBM Cloud yang dapat diakses melalui akun IBM Cloud. Pengguna perlu membuat instance untuk layanan STT dan TTS serta memperoleh kunci API untuk otentikasi.
- Memilih SDK yang tepat: IBM menyediakan SDK untuk berbagai bahasa pemrograman, termasuk Python, yang memudahkan integrasi layanan ini ke dalam aplikasi.
- Memahami API: Kenali dokumentasi API untuk STT dan TTS, karena pemahaman tentang metode, parameter, dan format respons yang tersedia sangat penting untuk integrasi yang efektif.
- Desain interaksi pengguna: Pertimbangkan cara pengguna berinteraksi dengan aplikasi melalui suara. Desain antarmuka suara yang lancar dan intuitif merupakan kunci keberhasilan aplikasi yang didukung suara.
- Mengelola data audio: Pastikan aplikasi Anda dapat menangkap data audio dengan benar untuk STT dan memproses output audio TTS yang kompatibel dengan antarmuka depan aplikasi.
- Privasi dan keamanan: Saat menangani data suara, terutama dalam aplikasi yang mungkin menangani informasi sensitif, penting untuk mempertimbangkan langkah-langkah privasi dan keamanan untuk melindungi data pengguna.

3.9 Masa depan teknologi yang didukung suara

Seiring dengan kemajuan teknologi STT dan TTS, potensi mereka untuk merevolusi interaksi manusia-komputer semakin besar. Pengembangan di masa depan mungkin akan berfokus pada peningkatan naturalitas dan ekspresivitas output TTS, serta peningkatan akurasi dan adaptabilitas STT dalam lingkungan kompleks seperti:

- Kecerdasan emosional: Sistem TTS masa depan mungkin akan dilengkapi dengan kecerdasan emosional yang lebih canggih, memungkinkan mereka menyesuaikan nada dan ekspresi berdasarkan konteks percakapan atau kondisi emosional pengguna, yang dideteksi melalui pola ucapan dan petunjuk linguistik.

- Pemahaman konteks: Peningkatan STT dengan pemahaman konteks yang lebih baik dapat menghasilkan transkripsi dan interpretasi ucapan yang lebih akurat, dengan mempertimbangkan konteks percakapan yang lebih luas atau domain aplikasi yang spesifik.
- Integrasi lintas platform: Seiring dengan semakin meluasnya teknologi berbasis suara, integrasi yang mulus antar platform dan perangkat akan menjadi krusial. Hal ini dapat memfasilitasi pengalaman pengguna yang lebih terpadu, memungkinkan interaksi yang berkelanjutan dan sadar konteks, terlepas dari perangkat atau aplikasi yang digunakan.

4 Generative AI-Powered Meeting Assistant

4.1 Teknologi yang Digunakan

- Automatic Speech Recognition (ASR): Teknologi yang mengubah bahasa lisan menjadi teks yang dapat dibaca.
- Large Language Models (LLMs): Model bahasa yang dapat memahami dan merangkum teks dengan efisien. Contoh yang digunakan adalah Llama 2 dari Meta.

4.2 IBM Watsonx.ai

IBM watsonx.ai adalah studio yang menyediakan alat terintegrasi untuk bekerja dengan generative AI dan membangun model machine learning. Fitur utama:

- Memungkinkan pengguna untuk melatih, menyetel, menerapkan, dan mengelola foundation models dengan mudah.
- Membantu dalam membangun aplikasi AI dengan lebih cepat dan efisien.

4.3 Produk dalam IBM Watsonx

- `watsonx.ai`: Studio untuk foundation models dan generative AI.
- `watsonx.data`: Tempat penyimpanan data.
- `watsonx.governance`: Toolkit untuk pemantauan dan tata kelola.

4.4 Alat dan Fitur dalam watsonx.ai

- Prompt Lab: Alat untuk bereksperimen dengan foundation models dan membangun prompts untuk berbagai tugas NLP (Natural Language Processing). Contoh Tugas: Question answering, content generation, text classification.
- Tuning Studio: Alat untuk menyesuaikan model berdasarkan data dan kebutuhan bisnis. Fungsi: Menggunakan metode tuning untuk meningkatkan performa model dengan sedikit contoh data.
- Pipeline Tool: Alat untuk mengelola dan mengotomatiskan siklus hidup model. Fungsi: Mengotomatiskan langkah-langkah dalam memuat data, melatih model, dan mengevaluasi model.

4.5 Pengenalan ke Meta Llama 2

Llama 2 adalah keluarga model bahasa besar (LLM) yang telah dilatih sebelumnya dan disesuaikan, yang dirilis oleh Meta AI. Model AI Llama 2 mampu melakukan berbagai tugas pemrosesan bahasa alami (NLP), mulai dari pembangkitan teks hingga kode pemrograman.

Model Llama asli merupakan tonggak penting dalam lanskap kecerdasan buatan generatif, menawarkan kemampuan untuk memahami dan menghasilkan teks yang mirip manusia. Model ini dirancang untuk mendukung berbagai aplikasi, mulai dari sistem tanya-jawab sederhana hingga tugas pemrosesan bahasa alami yang lebih kompleks.

Dengan membangun fondasi yang kokoh dari pendahulunya, seperti Generative Pretrained Transformer (GPT) dan Llama, Meta Llama 2 memperkenalkan peningkatan signifikan dalam memahami dan menghasilkan teks yang mirip manusia. Peningkatan ini berarti Meta Llama 2 dapat menangani lebih banyak data, menghasilkan respons yang lebih akurat dan koheren, serta melakukannya lebih cepat dan dengan sumber daya komputasi yang lebih sedikit dibandingkan model Llama asli. Peningkatan ini mendukung pemahaman yang lebih baik terhadap pertanyaan kompleks, pembangkitan teks yang lebih halus dan sesuai konteks, serta dukungan untuk berbagai bahasa yang lebih luas.

Dengan memanfaatkan kemajuan terbaru dalam riset dan teknologi AI, Meta Llama 2 bertujuan untuk melampaui kemampuan model sebelumnya seperti GPT-3, Bidirectional Encoder Representations from Transformers (BERT), dan Llama asli, menetapkan standar baru dalam efisiensi dan fleksibilitas untuk tugas pemrosesan bahasa alami.

4.5.1 Fitur Kunci dari Meta Llama 2

- Peningkatan pemahaman dan generasi teks: Llama 2 menunjukkan kemampuan pemahaman dan generasi teks yang superior, sehingga mampu menangani pertanyaan kompleks dan menghasilkan respons yang koheren dan relevan secara kontekstual.
- Dukungan multibahasa: Llama 2 menyediakan dukungan yang luas untuk berbagai bahasa, menjadikannya alat yang serbaguna untuk aplikasi global.
- Efisiensi dalam pemrosesan: Dengan optimasi pada arsitektur model, Llama 2 memproses informasi dengan lebih efisien, memungkinkan waktu respons yang lebih cepat bahkan untuk pertanyaan kompleks.
- Penyesuaian dan skalabilitas: Llama 2 dapat disesuaikan untuk tugas atau industri tertentu, memungkinkan bisnis dan peneliti menyesuaikan kemampuannya sesuai kebutuhan unik mereka.

4.5.2 Limitasi Meta Llama 2

Meskipun memiliki kelebihan, penerapan Llama 2 di tempat umum menimbulkan beberapa tantangan.

- Privasi dan keamanan data yang terganggu: Llama 2 adalah LLM publik yang mungkin tidak menawarkan tingkat privasi dan keamanan data yang sama pentingnya untuk menangani informasi sensitif atau kepemilikan. Menyoroti kekhawatiran seputar pelanggaran data atau akses tidak sah sangat penting untuk melindungi data rahasia, yang mungkin tidak dapat dilakukan dengan Llama 2.
- Kustomisasi terbatas: Sebagai LLM publik, pengguna Llama 2 mungkin memiliki kemampuan terbatas untuk menyesuaikan model dan keluarannya agar sesuai dengan kebutuhan bisnis tertentu atau persyaratan khusus domain, yang memengaruhi relevansi dan akurasi respons model.
- Berbagi sumber daya dan kinerja yang lambat: Llama biasanya digunakan bersama oleh banyak pengguna, yang menyebabkan kinerja dan waktu respons yang bervariasi, tidak cocok untuk aplikasi yang membutuhkan pemrosesan yang konsisten dan berkecepatan tinggi.
- Biaya tinggi: Llama 2 terkadang bisa lebih mahal daripada hosting pribadi untuk penggunaan volume tinggi atau komersial, terutama saat meningkatkan skala operasi.

4.6 Aplikasi dari Meta Llama 2

Kemampuan serbaguna Meta Llama 2 memungkinkan penggunaannya di berbagai bidang:

- Pembuatan konten: Llama 2 dapat membantu atau mengotomatisasi proses pembuatan konten, mulai dari penulisan artikel hingga pembuatan fiksi kreatif.
- Analisis dan ringkasan data: Ia dapat menganalisis volume teks yang besar dengan cepat dan memberikan ringkasan yang ringkas, mengekstrak wawasan kunci dari data.
- Terjemahan bahasa: Dengan kemampuan multibahasa, Llama 2 memfasilitasi terjemahan yang lancar antar bahasa, meningkatkan komunikasi antarbudaya.
- Alat pendidikan: Model ini dapat mendukung sistem bimbingan belajar, menawarkan pengalaman belajar yang dipersonalisasi dan penjelasan kepada siswa.

4.7 Wawasan Teknis terkait Meta Llama 2

Arsitektur Llama 2 didasarkan pada jaringan saraf transformer, jenis model pembelajaran mendalam yang telah merevolusi bidang pemrosesan bahasa alami. Dengan menggunakan mekanisme perhatian, Llama 2 dapat fokus pada bagian-bagian relevan dari data masukan, sehingga meningkatkan kemampuannya untuk memahami dan menghasilkan teks.

4.7.1 Contoh 1: Membuat teks berdasarkan prompt

Contoh ini menunjukkan cara membuat teks berdasarkan prompt menggunakan Meta Llama 2. Contoh ini mengasumsikan adanya paket Python untuk berinteraksi dengan Llama 2, mirip dengan cara Anda menggunakan model dari perpustakaan Hugging Face Transformers.

```
from meta_llama import MetaLlama2

# Initialize the model
model = MetaLlama2(model_name='meta-llama-2')

# Generate text based on a prompt
prompt = "What is the future of artificial intelligence?"
generated_text = model.generate_text(prompt)
print("Generated Text:", generated_text)
```

4.7.2 Contoh 2: Ringkasan data

Contoh ini menunjukkan bagaimana Meta Llama 2 dapat digunakan untuk meringkas teks yang lebih panjang menjadi ringkasan yang ringkas. Hal ini sangat berguna untuk mengekstrak informasi penting dari dokumen yang besar.

```
from meta_llama import MetaLlama2

# Initialize the model
model = MetaLlama2(model_name='meta-llama-2', task='summarization')

# Summarize text
text = """Artificial Intelligence (AI) has been a subject of
fascination and intensive research for decades. AI
technologies have evolved from basic algorithms to
advanced machine learning models, profoundly impacting
industries, healthcare, and everyday life. The future of
AI promises even more revolutionary changes, with
potential advancements in autonomous vehicles,
personalized medicine, and intelligent automation."""
summary = model.summarize(text)

print("Summary:", summary)
```

Note: In these examples, MetaLlama2 is a fictional class representing the interaction with the Meta Llama 2 model. In reality, interactions with such a model would depend on the specific API or library provided by the model's creators. The examples are designed to illustrate potential applications and should be adapted to the actual implementation details of Meta Llama 2 or similar models available through platforms like Hugging Face Transformers.

4.8 Pengenalan ke OpenAI Whisper

OpenAI Whisper adalah sistem pengenalan suara revolusioner yang dirancang untuk mentranskrip audio menjadi teks dengan akurasi tinggi. Dibangun berdasarkan model pembelajaran mendalam, Whisper menunjukkan kemampuan luar biasa dalam menangani berbagai jenis audio, mulai dari rekaman studio yang jernih hingga lingkungan yang bising, serta mendukung multiple bahasa. Fleksibilitas ini menjadikan Whisper alat yang sangat berharga bagi pengembang yang ingin mengintegrasikan fungsi suara ke dalam aplikasi mereka.

4.8.1 Fitur Kunci dari Whisper

- Akurasi tinggi: Akurasi Whisper merupakan salah satu fitur unggulannya, yang dicapai melalui pelatihan pada dataset yang beragam dan luas. Dataset ini mencakup berbagai jenis audio dan skenario, memungkinkan Whisper untuk menangani pola ucapan, aksen, dan dialek yang berbeda dengan presisi tinggi. Dengan dasar pembelajaran mendalam (deep learning), Whisper dapat memahami konteks, sehingga kemampuannya untuk mentranskrip dengan akurat tetap terjaga bahkan ketika kualitas audio buruk atau ucapan tidak jelas.
- Dukungan multibahasa: Whisper dapat mengenali dan mentranskrip ucapan dalam berbagai bahasa. Dilatih menggunakan sampel audio dari berbagai bahasa, Whisper mendukung transkripsi multibahasa tanpa memerlukan pemilihan bahasa secara manual. Hal ini membuatnya serbaguna untuk aplikasi global, memfasilitasi komunikasi dan pembuatan konten di berbagai komunitas linguistik.
- Ketahanan terhadap kebisingan: Keunggulan utama Whisper adalah ketahanannya dalam lingkungan berisik. Pelatihannya mencakup berbagai sampel audio berisik, yang membantu model secara efektif membedakan ucapan dari kebisingan latar belakang. Fitur ini sangat bermanfaat untuk aplikasi di kondisi akustik yang menantang, memastikan transkripsi yang andal dalam situasi mulai dari wawancara di jalan raya yang ramai hingga acara publik yang ramai.

4.9 Menyiapkan Whisper

Before diving into code examples and use cases, ensure your development environment is ready to use Whisper. You will need to have Python installed on

your system.

To install Whisper, consider the following command:

```
pip install git+https://github.com/openai/whisper.git
```

4.9.1 Contoh: Transkripsi Dasar dengan Whisper

Mari kita mulai dengan contoh sederhana tentang cara mentranskripsikan berkas audio menjadi teks menggunakan Whisper. Contoh ini mengasumsikan Anda memiliki berkas audio bernama `audio_example.mp3`.

```
import whisper

# Load the Whisper model
model = whisper.load_model("base")

# Transcribe the audio file
result = model.transcribe("audio_example.mp3")

# Output the transcription
print(result["text"])
```

Potongan kode ini memuat model dasar Whisper, mentranskripsikan berkas audio yang ditentukan, dan menampilkan transkripsi ke konsol.

4.9.2 Use Case Nyata

Whisper dapat diterapkan di berbagai bidang untuk meningkatkan aksesibilitas, efisiensi, dan pengalaman pengguna. Berikut adalah beberapa contoh penggunaan real-time:

- Subtitle otomatis untuk video: Tingkatkan aksesibilitas konten video online dengan secara otomatis menghasilkan subtitle dalam berbagai bahasa.
- Mesin pencari berbasis suara: Kembangkan mesin pencari yang memungkinkan pengguna melakukan pencarian menggunakan perintah suara, sehingga antarmuka menjadi lebih aksesibel dan ramah pengguna.
- Transkripsi rapat: Transkripsi rapat atau kuliah secara real-time, menyediakan catatan teks yang berharga yang dapat dicari dan dirujuk dengan mudah.
- Dukungan pelanggan multibahasa: Gunakan Whisper untuk mentranskripsi panggilan dukungan pelanggan, memfasilitasi dukungan dalam berbagai bahasa dan memudahkan analisis umpan balik pelanggan.

4.9.3 Fitur Tambahan

Kemampuan Whisper yang serbaguna memungkinkan penyesuaian dan optimasi sesuai dengan kebutuhan spesifik:

- Model bahasa: Whisper tersedia dalam berbagai ukuran (misalnya, tiny, base, small, medium, large). Model yang lebih besar menawarkan akurasi yang lebih tinggi tetapi memerlukan sumber daya komputasi yang lebih besar.
- Deteksi bahasa otomatis: Whisper dapat secara otomatis mendeteksi bahasa audio, memudahkan proses transkripsi untuk aplikasi multibahasa.

4.10 Mengintegrasikan Whisper ke dalam Aplikasi

API Python Whisper memudahkan integrasi ke dalam aplikasi. Misalnya, berikut ini adalah contoh kode untuk membuat layanan transkripsi berbasis web menggunakan Flask.

```
from flask import Flask, request
import whisper

app = Flask(__name__)
model = whisper.load_model("base")

@app.route("/transcribe", methods=["POST"])
def transcribe_audio():
    audio_file = request.files["audio"]
    result = model.transcribe(audio_file)
    return {"transcription": result["text"]}

if __name__ == "__main__":
    app.run(debug=True)
```

Aplikasi Flask sederhana ini mendefinisikan endpoint yang menerima berkas audio untuk transkripsi, menunjukkan cara Whisper dapat diintegrasikan ke dalam aplikasi web.

5 Summarize Your Private Data with Generative AI and RAG

5.1 Pengenalan LLMs dan Generative AI

- LLMs (Large Language Models): Model bahasa yang dilatih pada dataset besar untuk memahami dan menghasilkan teks. Mereka dapat membantu dalam merangkum dan mengekstrak informasi dari dokumen panjang.
- Generative AI: Teknologi yang dapat menghasilkan konten baru, seperti teks, gambar, atau suara, berdasarkan data yang telah dipelajari.

5.2 Proyek Chatbot

- Proyek ini melibatkan pembuatan chatbot yang memungkinkan pengguna mengunggah file PDF dan mengajukan pertanyaan tentang konten file tersebut.
- Flask: Framework web ringan untuk Python yang digunakan untuk mengembangkan backend chatbot, menangani permintaan HTTP dan memberikan respons.
- LangChain: Framework yang mengintegrasikan LLMs ke dalam aplikasi, memfasilitasi interaksi antara aplikasi Flask dan LLM.

5.3 Teknologi yang Digunakan

- Llama 2: Keluarga LLM yang dirilis oleh Meta, digunakan dalam proyek ini untuk memproses bahasa alami dan memahami input pengguna.
- RAG (Retrieval-Augmented Generation): Teknik yang memungkinkan Llama 2 mengakses informasi dari sumber eksternal untuk memberikan jawaban yang lebih akurat.

5.4 Tujuan dan Manfaat LangChain

LangChain membantu pengembang dalam mengintegrasikan LLM ke dalam aplikasi AI dengan menyediakan komponen dan antarmuka yang diperlukan. Manfaatnya sebagai berikut:

- Modularity: Memungkinkan pengembang untuk menyusun komponen seperti blok bangunan, sehingga mempercepat pengembangan.
- Extensibility: Memudahkan penambahan fitur baru dan integrasi dengan sistem eksternal.
- Decomposition: Memecah tugas kompleks menjadi langkah-langkah yang lebih kecil untuk menghasilkan respons yang relevan.

5.5 Cara Kerja LangChain

- Integrasi dengan Vector Databases: LangChain dapat terhubung dengan database vektor untuk pencarian semantik yang efisien.
- Pengolahan Data: Menggunakan metode untuk merespons permintaan kompleks dengan mengambil data dan menghasilkan ringkasan yang koheren.

5.6 Aplikasi Praktis LangChain

- Content Summarization: Meringkas artikel dan dokumen, membantu pengguna memahami dokumen hukum yang kompleks.
- Data Extraction: Mengambil statistik kunci dari laporan untuk menghasilkan wawasan yang dapat ditindaklanjuti.
- Question and Answer Systems: Meningkatkan layanan dukungan pelanggan dengan memberikan jawaban yang relevan berdasarkan konteks percakapan.
- Automated Content Generation: Membantu dalam penulisan otomatis seperti draf email dan dokumentasi teknis.

5.7 Pengenalan Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) adalah sebuah kerangka kerja yang menggabungkan kemampuan generasi teks dari LLM dengan pencarian informasi dari sumber eksternal untuk memberikan jawaban yang lebih akurat dan terkini.

5.8 Tantangan LLM

- No Source: LLM sering memberikan jawaban tanpa menyertakan sumber, yang dapat menyebabkan informasi yang tidak dapat dipercaya.
- Out of Date: Jawaban yang diberikan mungkin tidak lagi relevan atau akurat karena informasi yang sudah usang.

5.9 Solusi RAG

- Proses Pencarian: Dalam RAG, LLM pertama-tama mencari informasi dari content store (misalnya, internet atau dokumen tertentu) sebelum memberikan jawaban.
- Penggabungan Data: LLM menggabungkan informasi yang diambil dengan pertanyaan pengguna untuk menghasilkan jawaban yang lebih baik dan dapat dipertanggungjawabkan.

5.10 Manfaat RAG

- Update Informasi: Dengan menambahkan informasi baru ke dalam content store, LLM dapat memberikan jawaban yang lebih terkini tanpa perlu dilatih ulang.
- Sumber yang Dapat Dipercaya: LLM dapat memberikan bukti atau sumber untuk jawaban yang diberikan, mengurangi kemungkinan hallucination (membuat informasi yang tidak ada).

5.11 Manfaat menggunakan Llama 2 pribadi dengan RAG

Hosting Llama 2 secara privat dapat membantu dalam Retrieval-Augmented Generation (RAG).

Karena RAG adalah teknik yang menggabungkan kemampuan generatif LLM dengan pengambilan informasi untuk meningkatkan akurasi dan relevansi respons model dengan melakukan kueri pada basis data atau basis pengetahuan sebagai bagian dari proses pembangkitan, RAG memungkinkan model untuk menarik informasi eksternal yang mungkin belum dilatih secara eksplisit, sehingga menghasilkan respons yang lebih kaya kontekstual dan lebih presisi.

5.11.1 Alasan menggunakan Llama 2 pribadi untuk RAG

Pilihan untuk menggunakan Llama 2 yang dihosting secara privat untuk RAG didasarkan pada beberapa alasan.

- Keamanan dan privasi data yang ditingkatkan: Hosting privat memungkinkan kontrol yang lebih baik atas data yang sedang diproses, yang krusial untuk aplikasi yang menangani informasi sensitif atau kepemilikan.
- Kustomisasi yang ditingkatkan: Hosting privat menawarkan fleksibilitas untuk mengkustomisasi model dan basis pengetahuan terkaitnya, memastikan komponen pengambilan data disesuaikan dengan kebutuhan spesifik aplikasi.
- Optimalisasi kinerja yang difasilitasi: Hosting model secara privat memungkinkan pengoptimalan sumber daya dan infrastruktur komputasi untuk memenuhi persyaratan kinerja spesifik aplikasi.

5.12 LangChain untuk mengimplementasikan RAG

LangChain adalah kerangka kerja yang dirancang untuk menyederhanakan integrasi model bahasa dengan teknologi lain, seperti basis data dan basis pengetahuan, untuk aplikasi seperti RAG.

Kerangka kerja ini menyediakan alat dan abstraksi yang memudahkan pembangunan aplikasi NLP yang kompleks tanpa harus secara langsung mengelola kerumitan arsitektur model atau sistem pengambilan informasi.

5.12.1 Proses implementasi RAG dengan Llama 2

- Konfigurasi model: Ini melibatkan pengaturan Llama 2 di dalam LangChain dan penentuan parameter seperti ukuran model dan antarmuka untuk berinteraksi dengannya.
- Integrasi sumber data: Hubungkan model ke sumber data eksternal atau basis pengetahuan yang dapat digunakan untuk mengambil informasi selama proses pembuatan. LangChain mendukung berbagai jenis sumber data, memastikan fleksibilitas dalam mengintegrasikan pengetahuan.
- Pembuatan alur kerja: Menentukan alur kerja untuk menghasilkan respons, termasuk kueri pengambilan, memproses data yang diambil, dan menggabungkan informasi ini ke dalam proses generatif model.

Dengan mengabstraksi kompleksitas penggabungan model bahasa dengan sistem pengambilan data, LangChain membantu memaksimalkan potensi RAG dengan Llama 2, menciptakan aplikasi yang menghasilkan respons yang sangat akurat dan kaya informasi.

6 Babel Fish (Universal Language Translator) with LLM and SST TTS

7 Build and AI Career Coach